LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 5 – Relasi Antar Kelas



NAMA : Ahnaf Ariacheda

NIM : 10313240037

KELAS : PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Diagram Kelas (*Class Diagram*)

- Representasi visual berbasis UML yang memetakan struktur statis sistem, meliputi nama kelas, atribut, method, serta relasi antar kelas.
- Terbagi menjadi tiga kompartemen utama: baris pertama untuk nama kelas, baris kedua untuk atribut, dan baris ketiga untuk operasi atau method.
- Menggunakan simbol *access modifier* seperti tanda + untuk *public*, - untuk *private*, dan # untuk *protected*.
- Contoh penerapannya dapat dilihat pada kelas Mahasiswa yang memiliki atribut nim dan nama beserta method getter dan setter.

Hubungan Asosiasi (*Association*)

- Bentuk relasi paling longgar antara dua kelas terpisah yang saling berkomunikasi atau berinteraksi satu sama lain.
- Objek dari suatu kelas dapat memanggil *method* atau menggunakan data dari kelas lain tanpa harus memiliki kepemilikan struktural yang permanen.
- Contoh nyatanya adalah hubungan antara kelas Dosen dan kelas Mahasiswa, di mana dosen dapat memberikan nilai kepada mahasiswa melalui *method*, tetapi objek mahasiswa tersebut tidak menjadi atribut wajib di dalam kelas dosen.

Hubungan Agregasi (*Aggregation*)

- Relasi struktural yang merepresentasikan konsep *whole-part* atau keseluruhan dan bagian.
- Kelas kontainer bertindak sebagai keseluruhan yang menampung objek bagian di dalamnya menggunakan struktur *array*, namun kedua kelas tetap memiliki siklus hidup yang independen.
- Contoh penerapannya adalah kelas Jurusan yang menampung daftar objek Mahasiswa, di mana jika jurusannya dibubarkan, mahasiswa tersebut tidak ikut musnah.

Hubungan Komposisi (*Composition*)

- Bentuk khusus dari agregasi yang memiliki ikatan jauh lebih kuat dan mutlak karena objek bagian merupakan komponen fundamental.
- Objek bagian tidak bisa berdiri sendiri dan sepenuhnya bergantung pada kelas utama; jika kelas utama dimusnahkan, maka seluruh objek komponen di dalamnya akan ikut musnah.
- Contoh klasiknya adalah kelas Komputer yang tersusun atas komponen seperti CPU, Monitor, dan Mouse, di mana ketika komputer dihancurkan, komponen-komponen di dalamnya ikut tidak berfungsi.

Hubungan Generalisasi (*Generalization*)

- Hubungan hierarkis antar kelas yang berkaitan erat dengan konsep pewarisan atau *inheritance*.
- Kelas turunan (*subclass*) dapat mewarisi seluruh atribut serta *method* dari kelas induknya (*superclass*) untuk mendukung penggunaan kembali kode.
- Contoh penerapannya adalah kelas Mobil dan Motor sebagai turunan yang mewarisi sifat-sifat umum dari kelas induk Kendaraan.

B. Guided

1. Package agregasi >> Class Mahasiswa.java

```java
package com.nao.guided.agregasi;

public class Mahasiswa {
    private String NIM, Nama;

    public Mahasiswa(String NIM, String Nama){
        this.NIM = NIM;
        this.Nama = Nama;
    }

    //getter
    public String getNIM(){
        return NIM;
    }

    public String getNama(){
        return Nama;
    }

    //setter
    public void setNIM(String NIM){
        this.NIM = NIM;
    }

    public void setNama(String Nama){
        this.Nama = Nama;
```

```
    }

}
```

Class Jurusan.java

```
package com.nao.guided.agregasi;

public class Jurusan {
    private static int jumlahMahasiswa;
    private String kodeJurusan, namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    //constructor
    public Jurusan(String kodeJurusan, String namaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
    }

    //behavior
    public void addMahasiswa(Mahasiswa m){
        this.daftarMahasiswa[jumlahMahasiswa] = m;
        jumlahMahasiswa++;
    }

    public void displayMahasiswa(){
        System.out.println("Kode Jurusan: " + this.kodeJurusan);
        System.out.println("Nama Jurusan: " + this.namaJurusan);

        System.out.println("Daftar Mahasiswa: ");

        for(int i=0; i<jumlahMahasiswa; i++){
            System.out.println(i+1 +  ". " + daftarMahasiswa[i].getNIM() + "  " +
daftarMahasiswa[i].getNama());
        }
    }
    //getter
    public String getKodeJurusan(){
```

```
        return kodeJurusan;
    }

    public String getNamaJurusan(){
        return namaJurusan;
    }

    //setter
    public void setKodeJurusan(String kodeJurusan){
        this.kodeJurusan = kodeJurusan;
    }

    public void setNamaJurusan(String namaJurusan){
        this.namaJurusan = namaJurusan;
    }
}
```

Output :

```
Kode Jurusan: SD
Nama Jurusan: Sains Data
Daftar Mahasiswa:
1. 103112400148  Rendhi
2. 103112400149  John
```

2. **Package asosiasi >> Class Mahasiswa.java**

```
package com.nao.guided.asosiasi;


public class Mahasiswa {
    private String nim;
    private String nama;
    private int nilai;

    public void setNama(String nama){
        this.nama = nama;
    }

    public void setNim(String nim){
        this.nim = nim;
    }

    public void setNilai(int nilai){
        this.nilai = nilai;
    }

    //getter
public int getNilai(){
        return this.nilai;
    }

    public String getNama(){
```

```
        return this.nama;
    }
    public String getNim(){
        return this.nim;
    }

}
```

Class Dosen.java

```
package com.nao.guided.asosiasi;

public class Dosen {
    private String kodeDosen;
    private String namaDosen;

    //Setter
    public void setKodeDosen(String kodeDosen){
        this.kodeDosen = kodeDosen;
    }
    public void setNamaDosen(String namaDosen){
        this.namaDosen = namaDosen;
    }

    //Getter
    public String getKodeDosen(){
        return this.kodeDosen;
    }

    public String getNamaDosen(){
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai){
```

```
        s.setNilai(nilai);
    }
    public int getScore(Mahasiswa s){
        return s.getNilai();
    }

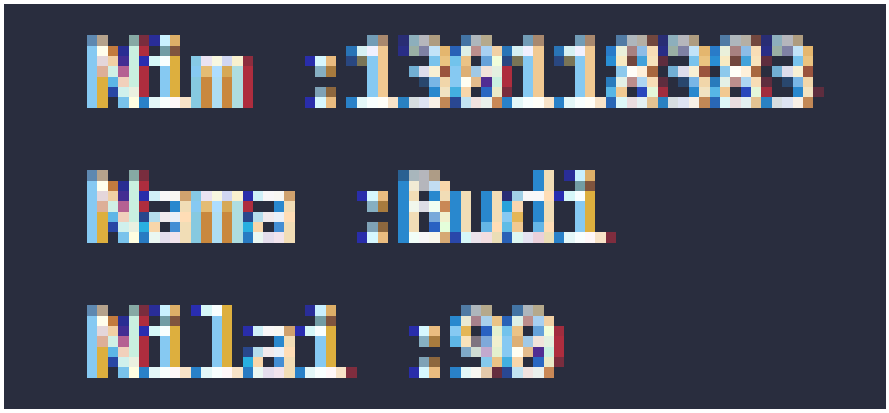
    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();

        m.setNim("130118383");
        m.setNama("Budi");

        Dosen d = new Dosen();
        d.giveScore(m, 90);

        System.out.println("Nim :"+m.getNim());
        System.out.println("Nama :"+m.getNama());
        System.out.println("Nilai :"+d.getScore(m));
    }
}
```

Output :



3. Package komposisi >> Class Komputer.java

```
package com.nao.guided.komposisi;

public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String brandMonitor, String brandMouse, String
tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }
}
```

Class Monitor.java

```
package com.nao.guided.komposisi;

public class Monitor {
    private String brand;

    public Monitor(String brand){
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
    }
```

```
}
```

Class Mouse.java

```
package com.nao.guided.komposisi;

public class Mouse {
    private String brand, tipe;

    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

Class Processor.java

```
package com.nao.guided.komposisi;

public class Processor {
    private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
```

```
    }
}
```

Output :

```
=== SPESIFIKASI KOMPUTER ===
Intel Core i5, 3200
LG 24 Inch
Logitech, Wireless
```

Class Main.java (untuk run package agregasi dan komposisi)

```java
package com.nao;

import com.nao.guided.agregasi.Jurusan;
import com.nao.guided.agregasi.Mahasiswa;
import com.nao.guided.komposisi.Komputer;

public class Main {
    public static void main(String[] args) {
        runGuided2();
        runGuided3();
    }

    public static void runGuided1(){}

    public static void runGuided2(){
        Mahasiswa m1 = new Mahasiswa("103112400148", "Rendhi");
        Mahasiswa m2 = new Mahasiswa("103112400149", "John");

        Jurusan prodi = new Jurusan("SD","Sains Data");

        prodi.addMahasiswa(m1);
```

```
        prodi.addMahasiswa(m2);

        prodi.displayMahasiswa();
    }

    public static void runGuided3(){
        Komputer pc = new Komputer("Intel Core i5",3200,"LG 24 Inch","Logitech","Wireless");
        pc.displaySpecKomputer();
    }
}
```

C. Unguided

1. Soal Unguided — Modul 5: Pemrograman Berorientasi Objek

```
# Soal Unguided – Modul 5: Pemrograman Berorientasi Objek
### Relasi Antar Kelas: Komposisi, Agregasi, dan Asosiasi

---

## Petunjuk Umum

Buatlah program Java berdasarkan cerita berikut. Rancang sendiri struktur program yang diperlukan sebelum menuliskan kode.

- Tentukan sendiri field, constructor, dan method yang diperlukan, mengikuti hint dan tugas yang disediakan.
- Data di dalam object tidak boleh diakses atau diubah langsung dari luar class-nya (gunakan enkapsulasi).
- Gunakan nama parameter yang sama dengan nama field pada constructor (sehingga `this` perlu dipakai secara eksplisit).
- Proyek wajib mengikuti konvensi penamaan standar (PascalCase untuk Class, camelCase untuk variabel/method).
- Gunakan array standar bertipe tetap (bukan `ArrayList` atau koleksi library lainnya).
- Tidak perlu menggunakan inheritance, overriding, maupun abstract class.

---
```

Soal — Sistem Manajemen Pesanan Cafe

Sebuah cafe membutuhkan sistem sederhana untuk mencatat dan memproses pesanan pelanggan. Dalam sistem ini, relasi antar kelas dirancang untuk mencerminkan kondisi nyata operasional cafe.

Pengerjaan soal ini dibagi menjadi 3 bagian tugas utama yang masing-masing melatih satu jenis relasi antar kelas, diikuti dengan pengujian pada kelas utama (`Main`).

Tugas 1: Implementasi Komposisi (Composition)

Komposisi adalah relasi di mana sebuah objek memiliki objek lain sebagai bagian internalnya yang **tidak dapat berdiri sendiri**. Siklus hidup objek internal terikat penuh pada objek utamanya.

Pada sistem cafe:
- Setiap item pesanan (`ItemPesanan`) pasti memiliki rincian harga (`RincianHarga`) yang mencatat harga dasar dan pajak (10%).
- Objek `RincianHarga` harus di-instansiasi langsung di dalam constructor `ItemPesanan`. Objek `RincianHarga` tidak pernah dibuat di `main`. Jika `ItemPesanan` dihapus, maka `RincianHarga` miliknya ikut hilang.

Kebutuhan Tugas 1

Buatlah dua class: `RincianHarga` dan `ItemPesanan`.

Field yang disarankan:

Class	Field	Jenis	Tipe	Keterangan
`RincianHarga`	`hargaDasar`	Instance field	`double`	Harga awal sebelum pajak
`RincianHarga`	`pajak`	Instance field	`double`	Nilai nominal pajak (10% dari harga dasar)
`ItemPesanan`	`namaItem`	Instance field	`String`	Nama menu (misal: "Iced Americano")
`ItemPesanan`	`rincian`	Instance field	`RincianHarga`	Objek internal rincian harga (**Komposisi**)

Method yang disarankan:

Class	Method	Return	Keterangan
`RincianHarga`	`getHargaTotal()`	`double`	Mengembalikan nilai `hargaDasar + pajak`
`ItemPesanan`	`getNamaItem()`	`String`	Mengembalikan nama item pesanan
`ItemPesanan`	`getRincian()`	`RincianHarga`	Mengembalikan objek rincian harga

Tugas 2: Implementasi Agregasi (Aggregation)

Agregasi adalah relasi di mana sebuah objek menampung kumpulan objek lain yang **dapat berdiri sendiri**. Objek yang ditampung dibuat secara independen di luar, lalu dimasukkan ke dalam objek wadah.

Pada sistem cafe:
- Sebuah `Pesanan` menampung beberapa objek `ItemPesanan` menggunakan array standar.
- Objek `ItemPesanan` dibuat terpisah terlebih dahulu (di `main`), lalu dimasukkan satu per satu ke dalam `Pesanan` melalui method `tambahItem()`. Jika `Pesanan` dibatalkan atau dihapus, objek `ItemPesanan` tersebut secara konsep tetap ada.

Kebutuhan Tugas 2

Buatlah class `Pesanan`.

Field yang disarankan:

Field	Jenis	Tipe	Keterangan
`nomorMeja`	Instance field	`int`	Nomor meja pelanggan
`daftarItem`	Instance field	`ItemPesanan[]`	Array penampung item pesanan (**Agregasi**)
`jumlahItem`	Instance field	`int`	Penanda jumlah item yang sudah masuk ke array

Method yang disarankan:

Method	Return	Keterangan
`tambahItem(ItemPesanan item)`	`void`	Memasukkan objek `ItemPesanan` dari luar ke dalam array `daftarItem`

```
| `tampilkanPesanan()` | `void` | Menampilkan nomor meja dan seluruh item beserta harga total per item |
| `getNomorMeja()` | `int` | Mengembalikan nomor meja |
| `getDaftarItem()` | `ItemPesanan[]` | Mengembalikan array daftar item |
| `getJumlahItem()` | `int` | Mengembalikan jumlah item yang tersimpan |

---

## Tugas 3: Implementasi Asosiasi (Association)

Asosiasi adalah relasi interaksi antara dua objek independen yang **saling menggunakan tetapi tidak saling memiliki**. Objek tidak menyimpan objek lain sebagai field permanen, melainkan berinteraksi melalui parameter method.

Pada sistem cafe:
- Seorang `Kasir` memproses pembayaran dari sebuah `Pesanan`.
- Class `Kasir` tidak menyimpan objek `Pesanan` sebagai field, melainkan menerima objek `Pesanan` sebagai parameter pada method `prosesPembayaran()`.

### Kebutuhan Tugas 3

Buatlah class `Kasir`.

**Field yang disarankan:**

| Field | Jenis | Tipe | Keterangan |
|---|---|---|---|
| `namaKasir` | Instance field | `String` | Nama petugas kasir |

**Method yang disarankan:**

| Method | Return | Keterangan |
|---|---|---|
| `prosesPembayaran(Pesanan pesanan)` | `void` | Mengakses data `Pesanan`, menghitung akumulasi total bayar seluruh item, lalu mencetak struk pembayaran (**Asosiasi**) |

---

## Tugas 4: Eksekusi dan Pengujian di `Main`
```

Buatlah class `Main` untuk merangkai seluruh kelas dan menguji ketiga jenis relasi.

Alur program di `main`

1. Buat 1 objek `Kasir` (misal: `"Budi"`).
2. Buat 3 objek `ItemPesanan` secara independen:
   - `"Iced Americano"`, harga dasar: `18000`
   - `"Beef Croissant"`, harga dasar: `25000`
   - `"Matcha Latte"`, harga dasar: `22000`
3. Buat 1 objek `Pesanan` untuk **Meja 4** (kapasitas array misal: 5 item).
4. Masukkan ketiga objek `ItemPesanan` ke dalam objek `Pesanan` menggunakan method `tambahItem()`.
5. Tampilkan rincian pesanan dengan memanggil `tampilkanPesanan()` pada objek `Pesanan`.
6. Jalankan pemrosesan pembayaran dengan memanggil method `prosesPembayaran(pesanan)` dari objek `Kasir`.

Output yang Diharapkan

```
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
```

```
## Yang Perlu Diperhatikan Saat Mengerjakan

- **Komposisi:** Objek `RincianHarga` wajib di-instansiasi di dalam constructor `ItemPesanan` menggunakan kata kunci `new`. Dilarang melakukan `new RincianHarga(...)` di dalam `main`.
- **Agregasi:** Objek `ItemPesanan` wajib di-instansiasi di `main` terlebih dahulu, kemudian dikirimkan sebagai argumen ke method `tambahItem()`.
- **Asosiasi:** Class `Kasir` dilarang memiliki field bertipe `Pesanan`. Interaksi hanya terjadi saat method `prosesPembayaran(Pesanan pesanan)` dipanggil.
```

Jawaban :

- **Class RincianHarga.java**

```java
package com.nao.unguided;


public class RincianHarga {

    //field (atribut) hargaDasar dan pajak
    private double hargaDasar;
    private double pajak;


    //constructor untuk menginisialisasi hargaDasar dan pajak
    public RincianHarga(double hargaDasar, double pajak){
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 0.10;
    }


    //method untuk menghitung harga total (hargaDasar + pajak)
    public double getHargaTotal(){
        return this.hargaDasar + this.pajak;
    }
```

```
    //method getter untuk hargaDasar untuk cetak struk
    public double getHargaDasar(){
        return this.hargaDasar;
    }

    //mengambil nominal pajak untuk cetak struk
    public double getPajak(){
        return this.pajak;
    }
}
```

- **Class ItemPesanan.java**

```
package com.nao.unguided;

//field (atribut) namaItem dan rincian
public class ItemPesanan {
    private String namaItem;
    private RincianHarga rincian;

    //constructor untuk minta nama menu dan harga dasar saat objek dibuat
    public ItemPesanan(String namaItem, double hargaDasar){
    //enkapsulasi dan parameter this.
        this.namaItem = namaItem;
    //buat objek di dalam constructor untuk menginisialisasi rincian harga
        this.rincian = new RincianHarga(hargaDasar);
    }

    //method untuk return nama item
    public String getNamaItem(){
        return this.namaItem;
    }
```

```
    //method untuk return objek rincian harga(harga dasar dan pajak)
    public RincianHarga getRincian(){
        return this.rincian;
    }
}
```

- **Class Pesanan.java**

```
package com.nao.unguided;

public class Pesanan {
    //field (atribut) nomorMeja, daftarItem, dan jumlahItem
    private int nomorMeja;
    private ItemPesanan[] daftarItem;
    private int jumlahItem;

    //constructor untuk inisiasi nomorMeja dan daftarItem
    public Pesanan(int nomorMeja, int kapasitas){
        this.nomorMeja = nomorMeja;
        //buat array kosong untuk menampung daftar item pesanan
        this.daftarItem = new ItemPesanan[kapasitas];
        //mulai dari 0 karena belum ada item yang dipesan
        this.jumlahItem = 0;
    }

    //method untuk add item
    //karena parameter item itu objek yang sudah jadi dari class ItemPesanan, maka kita bisa langsung
kirim ke sini
    public void tambahItem(ItemPesanan item){
        //taruh objek item di array pada index jumlahItem
        this.daftarItem[this.jumlahItem] = item;
        //tambah 1 biar orderan masuk ke index berikutnya
        this.jumlahItem++;
    }

    //method buat show pesanan
```

```java
    public void tampilkanPesanan(){
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + this.nomorMeja);

        //loop sebanyak itemm yang dah masuk
        for (int i = 0; i < this.jumlahItem; i++){
            //ambil objek orderan ke-i dari array daftarItem
            ItemPesanan item = this.daftarItem[i];

            //ambil data berantai dari item > rincian > metode angka
            double hargaDasar = item.getRincian().getHargaDasar();
            double pajak = item.getRincian().getPajak();
            double total = item.getRincian().getHargaTotal();

            //format print
            System.out.println((i + 1) + ". " + item.getNamaItem() + " - Rp " + total + " (Harga: " +
hargaDasar + ", Pajak: " + pajak + ")");
        }
        System.out.println();
    }

    //getter dipake kasir buat ambil data pesanan
    public int getNomorMeja(){
        return this.nomorMeja;
    }

    public ItemPesanan[] getDaftarItem(){
        return this.daftarItem;
    }

    public int getJumlahItem(){
        return this.jumlahItem;
    }
}
```

- **Class Kasir.java**

```java
package com.nao.unguided;

public class Kasir {
    private String namaKasir;

    //constructor untuk inisiasi nama kasir
    public Kasir(String namaKasir){
        this.namaKasir = namaKasir;
    }

    //method untuk menampilkan struk pesanan
    //objek 'pesanan' mampir bentar buat diitung
    public void prosesPembayaran(Pesanan pesanan){
        //siapin dompet kosong
        double totalSubtotal = 0;
        double totalPajak = 0;

        //ambil arrray list item dari orderan yang dipinjem
        ItemPesanan[] items = pesanan.getDaftarItem();

        //loop untuk itung total semua harga yang di cart
        for (int i = 0; i < pesanan.getJumlahItem(); i++){
            //ambil harga dasar dari item ke-i, tambah ke subtotal
            totalSubtotal += items[i].getRincian().getHargaDasar();
            //ambil pajak dari item ke-i, tambah ke total pajak
            totalPajak += items[i].getRincian().getPajak();
        }

        //itung grand total (subtotal + pajak)
        double totalBayar = totalSubtotal + totalPajak;

        //cetak struk sesuai format
        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + this.namaKasir);
        System.out.println("Pesanan        : Meja " + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " + pesanan.getJumlahItem());
```

```
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("==============================");
    }
}
```

- **Class Main.java**

```
package com.nao.unguided;

public class Main {
    public static void main(String[] args){
        //siapin kasir
        Kasir kasirBudi = new Kasir("Budi");

        //siapin item yang diorder
        ItemPesanan kopi = new ItemPesanan("Iced Americano", 18000.0);
        ItemPesanan roti = new ItemPesanan("Beef Croissant", 25000.0);
        ItemPesanan teh = new ItemPesanan("Matcha Latte", 22000.0);

        //siapin nampan untuk meja no 4 (kapasitass di set 5)
        Pesanan pesananMeja4 = new Pesanan(4,5);

        //masukin barang ke nampan (agregaasi)
        pesananMeja4.tambahItem(kopi);
        pesananMeja4.tambahItem(roti);
        pesananMeja4.tambahItem(teh);

        //show list orderan sementara
        pesananMeja4.tampilkanPesanan();

        //kasihh nampan ke kasir buat diitung totalnya
        kasirBudi.prosesPembayaran(pesananMeja4);
    }
}
```

Output :

```
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
```

Penjelasan :

Program Sistem Manajemen Pesanan Cafe ini menggunakan 5 kelas yang saling berinteraksi untuk mengimplementasikan tiga pilar relasi antar objek:

- Komposisi (ItemPesanan berelasi dengan RincianHarga): Merupakan relasi ketergantungan mutlak. Korelasinya: Kelas ItemPesanan secara otomatis menciptakan (meng-instansiasi) objek RincianHarga secara internal di dalam constructor-nya. Objek rincian harga tidak akan pernah ada jika item pesanan tidak dibuat.
- Agregasi (Pesanan berelasi dengan ItemPesanan): Merupakan relasi wadah dan isi yang saling independen. Korelasinya: Kelas Pesanan bertindak sebagai wadah (menggunakan tipe data array). Objek ItemPesanan dibuat secara terpisah di luar kelas, lalu dikirim masuk ke dalam objek Pesanan melalui parameter method. Jika pesanan dibatalkan, objek item secara konsep tetap bisa eksis.
- Asosiasi (Kasir berelasi dengan Pesanan): Merupakan relasi interaksi sementara tanpa adanya kepemilikan. Korelasinya: Kelas Kasir tidak menyimpan kelas Pesanan sebagai atribut permanen (field). Interaksi hanya terjadi saat method pembayaran dijalankan, di mana objek Pesanan dikirimkan (dipinjamkan) melalui parameter agar kasir dapat membaca isinya, menghitung total biaya, lalu mencetak struk.
- Pusat Eksekusi (Main): Merupakan dirigen yang mengatur semua alur. Korelasinya: Kelas ini merangkai seluruh objek dengan cara membuat objek-objek mandiri, memicu

terjadinya Agregasi dengan memasukkan item ke dalam wadah pesanan, dan menginisiasi Asosiasi dengan menyerahkan wadah pesanan tersebut kepada kasir untuk diproses.

D. Kesimpulan

Berdasarkan praktikum Modul 5 mengenai Relasi Antar Kelas, dapat disimpulkan bahwa:

1. Dalam Object Oriented Programming (OOP), kelas-kelas dapat saling berkomunikasi dan berinteraksi membentuk suatu kesatuan sistem melalui mekanisme relasi.
2. **Asosiasi** merupakan bentuk hubungan dasar antara dua kelas yang independen di mana satu kelas menggunakan fungsionalitas atau data dari kelas lain (contohnya melalui passing parameter method), tanpa menjadikannya sebagai atribut permanen.
3. **Agregasi** merupakan relasi berwujud "keseluruhan-bagian" (whole-part) yang sifat ikatannya longgar. Objek bagian dapat diciptakan secara independen di luar kelas wadahnya dan tetap eksis meskipun objek wadahnya dihancurkan.
4. **Komposisi** merupakan relasi berwujud "keseluruhan-bagian" dengan ikatan yang sangat kuat, di mana eksistensi kelas bagian (part) bergantung sepenuhnya pada kelas keseluruhan (whole). Objek bagian baru akan diciptakan bersamaan dengan diciptakannya objek utama.
5. Penggunaan ketiga relasi ini memungkinkan perancangan program Java yang lebih modular, terstruktur, serta mampu mencerminkan skenario hubungan objek secara logis sesuai dengan kondisi di dunia nyata.